

AWS WordPress Hosting

1. Objective

Description

1. Launch a AWS instances (OS can choose by your choice)
2. Host WordPress files under a user directory
3. Create an RDS instance
4. Connect RDS Database with your WordPress and run webpage using URL myawshosting.net

NB: Only Minimum access is granted

2. EC2 — Instance Creation

2.1 Launch the EC2 Instance

A RHEL 10.2 (Coughlan) EC2 instance was launched in AWS.

The instance was accessed using the ec2-user account through SSH.

2.2 Configure EC2 Security Group

The EC2 security group was configured with the required minimum access.

Rules included:

- SSH — TCP 22 — allowed from the required source.
- HTTP — TCP 80 — allowed for web access.
- MySQL port 3306 was not opened on the EC2 security group.

3. Install Apache and PHP

3.1 Install Required Packages

Apache, PHP and the required PHP extensions were installed using:

```
sudo dnf install -y httpd php php-mysqlnd php-gd php-mbstring php-xml php-json php-curl unzip wget
```

3.2 Start and Enable Apache

Apache was started and configured to start automatically:

```
sudo systemctl enable --now httpd
```

3.3 Verify Apache

The Apache service was checked:

```
sudo systemctl status httpd
```

Apache was successfully running on the EC2 instance.

4. Install and Host WordPress

4.1 Create WordPress User Directory

The WordPress files were hosted under the ec2-user home directory.

The directory was created:

```
mkdir -p /home/ec2-user/wordpress
```

4.2 Download WordPress

WordPress was downloaded from the official WordPress website:

```
cd /home/ec2-user/wordpress  
wget https://wordpress.org/latest.tar.gz
```

The archive was extracted:

```
tar -xzf latest.tar.gz
```

The WordPress files were placed directly under:

```
/home/ec2-user/wordpress
```

The directory contained files and directories such as:

```
index.php
wp-admin/
wp-content/
wp-includes/
wp-config-sample.php
```

4.3 Configure Permissions

The home directory was made accessible to Apache:

```
sudo chmod 755 /home/ec2-user
```

WordPress permissions were configured:

```
sudo chmod -R 755 /home/ec2-user/wordpress
```

4.4 Configure Apache Virtual Host

An Apache configuration file was created:

```
/etc/httpd/conf.d/wordpress.conf
```

The configuration used was:

```
<VirtualHost *:80>
    ServerName myawshosting.net

    DocumentRoot /home/ec2-user/wordpress

    <Directory /home/ec2-user/wordpress>
        AllowOverride All
        Require all granted
    </Directory>

    DirectoryIndex index.php index.html
</VirtualHost>
```

4.5 Configure SELinux

Because WordPress was hosted inside `/home/ec2-user/wordpress`, SELinux file contexts were configured.

```
sudo semanage fcontext -a -t httpd_sys_content_t "/home/ec2-user/wordpress(/.*)?"
```

The SELinux context was applied:

```
sudo restorecon -Rv /home/ec2-user/wordpress
```

4.6 Verify Apache Configuration

Apache configuration was tested:

```
sudo apachectl configtest
```

The configuration was successfully validated and Apache was restarted.

```
sudo systemctl restart httpd
```

4.7 Verify WordPress

WordPress was tested locally:

```
curl -I http://localhost
```

The response showed that Apache and PHP were processing WordPress:

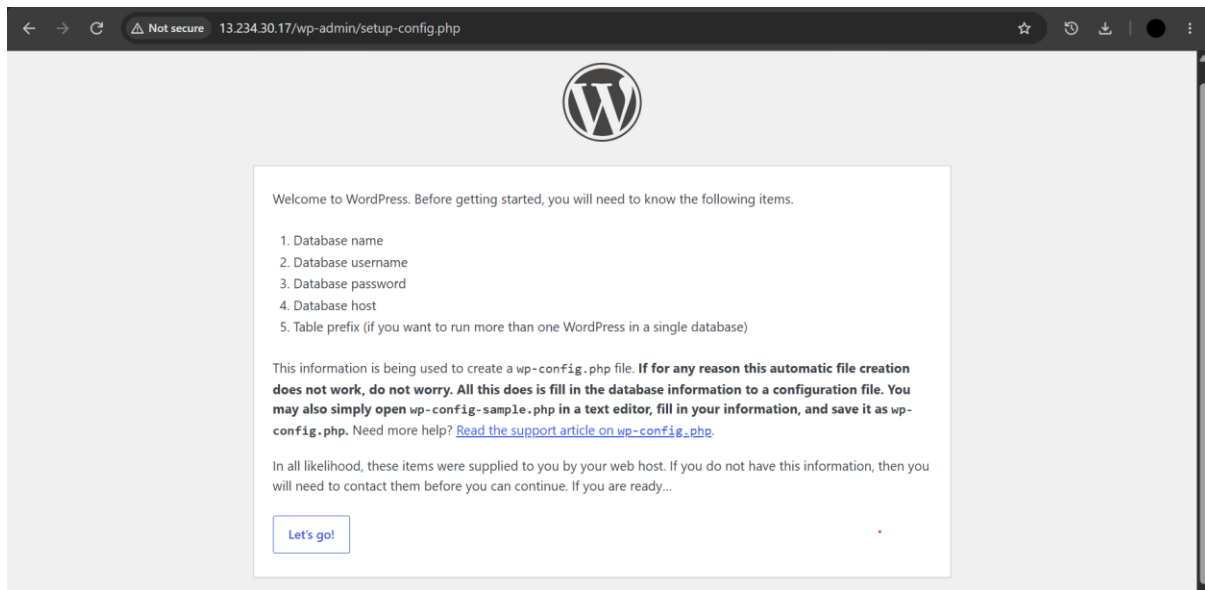
HTTP/1.1 302 Found

Server: Apache/2.4.63 (Red Hat Enterprise Linux)

X-Powered-By: PHP/8.3.33

Location: <http://localhost/wp-admin/setup-config.php>

This confirmed that Apache, PHP and WordPress were working correctly.



5. Amazon RDS — MySQL Database

5.1 Create the RDS Instance

An Amazon RDS MySQL database was created.

The database instance used:

- **Engine:** MySQL
- **DB Identifier:** database-1
- **Master Username:** admin
- **Port:** 3306
- **Public Access:** No

The RDS instance reached the:

Available

state.

5.2 Configure RDS Security Group

The RDS security group was configured to allow MySQL traffic only from the EC2 security group.

Rule:

Type: MySQL/Aurora
Protocol: TCP
Port: 3306
Source: EC2 Security Group

The database was **not exposed to the public internet**.

5.3 Verify RDS

The RDS endpoint was obtained:

`database-1.c1kyeyw6ujt6.ap-south-1.rds.amazonaws.com`

The RDS instance was available and ready for connection.

Output / Screenshot

- RDS instance showing Available.
- RDS endpoint.
- RDS security group configuration showing EC2 security group as the source.

6. Connect EC2 to RDS

6.1 Install MySQL Client

The MySQL client package available on RHEL 10.2 was identified as:

`mysql8.4`

It was installed using:

```
sudo dnf install -y mysql8.4
```

6.2 Connect to RDS

The EC2 instance was connected to the private RDS database:

```
mysql -h database-1.c1kyeyw6ujt6.ap-south-1.rds.amazonaws.com -P  
3306 -u admin -p
```

The connection was successful.

6.3 Verify Databases

The available databases were checked:

```
SHOW DATABASES;
```

Initially, the standard MySQL databases were displayed.

6.4 Create WordPress Database

A dedicated database for WordPress was created:

```
CREATE DATABASE wordpress;
```

The database was then available for WordPress.

```
mysql> CREATE DATABASE wordpress;
Query OK, 1 row affected (0.05 sec)

mysql> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sys |
| wordpress |
+-----+
5 rows in set (0.00 sec)

mysql> EXIT;
Bye
[ec2-user@ip-172-31-3-224 ~]$
```

7. Configure WordPress with RDS

7.1 Create WordPress Configuration

The WordPress configuration file was created from the sample file:

```
cd /home/ec2-user/wordpress
cp wp-config-sample.php wp-config.php
```

7.2 Configure Database Credentials

The following values were configured in `wp-config.php`:

```
define( 'DB_NAME', 'wordpress' );
define( 'DB_USER', 'admin' );
define( 'DB_PASSWORD', 'YOUR_RDS_PASSWORD' );
define( 'DB_HOST', 'database-1.c1kyeyw6ujt6.ap-south-
1.rds.amazonaws.com' );
```

The RDS endpoint was used as the WordPress database host.

7.3 Configure SELinux Database Access

Although the EC2 instance could connect to RDS using the MySQL client, Apache/PHP was initially prevented by SELinux from making database connections.

The SELinux status was checked:

```
getsebool httpd_can_network_connect_db
```

The boolean was found to be disabled.

It was enabled permanently:

```
sudo setsebool -P httpd_can_network_connect_db 1
```

The setting was verified:

```
getsebool httpd_can_network_connect_db
```

Expected result:

```
httpd_can_network_connect_db --> on
```

7.4 Complete WordPress Installation

After enabling the required SELinux permission, the WordPress installation page was opened.

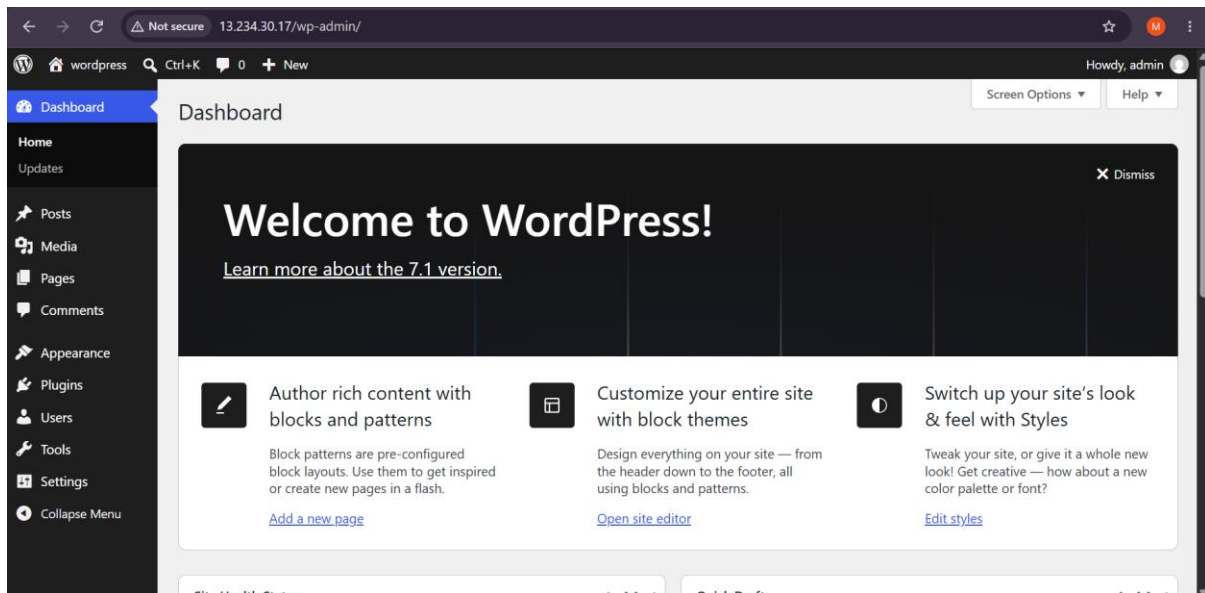
The WordPress installation was completed and the WordPress dashboard became accessible.

7.5 Verify WordPress Dashboard

The WordPress dashboard was successfully displayed.

This confirmed that:

- WordPress was running on EC2.
- PHP was working.
- WordPress could communicate with RDS.
- The RDS MySQL database was being used by WordPress.



8. Configure myawshosting.net

8.1 Configure Apache Server Name

The Apache virtual host was configured with:

`ServerName myawshosting.net`

8.2 Configure Windows Hosts File

Because the hostname was being used as a lab/test domain, the Windows hosts file was configured.

The file:

`C:\Windows\System32\drivers\etc\hosts`

was updated with:

`13.234.30.17 myawshosting.net`

This mapped the hostname to the EC2 public IP on the local Windows machine.

8.3 Flush DNS Cache

The Windows DNS cache was flushed:

```
ipconfig /flushdns
```

8.4 Verify HTTP Connectivity

The hostname was tested using:

```
curl -I http://myawshosting.net
```

The response was:

```
HTTP/1.1 200 OK
```

```
Server: Apache/2.4.63 (Red Hat Enterprise Linux)
```

```
X-Powered-By: PHP/8.3.33
```

```
Content-Type: text/html; charset=UTF-8
```

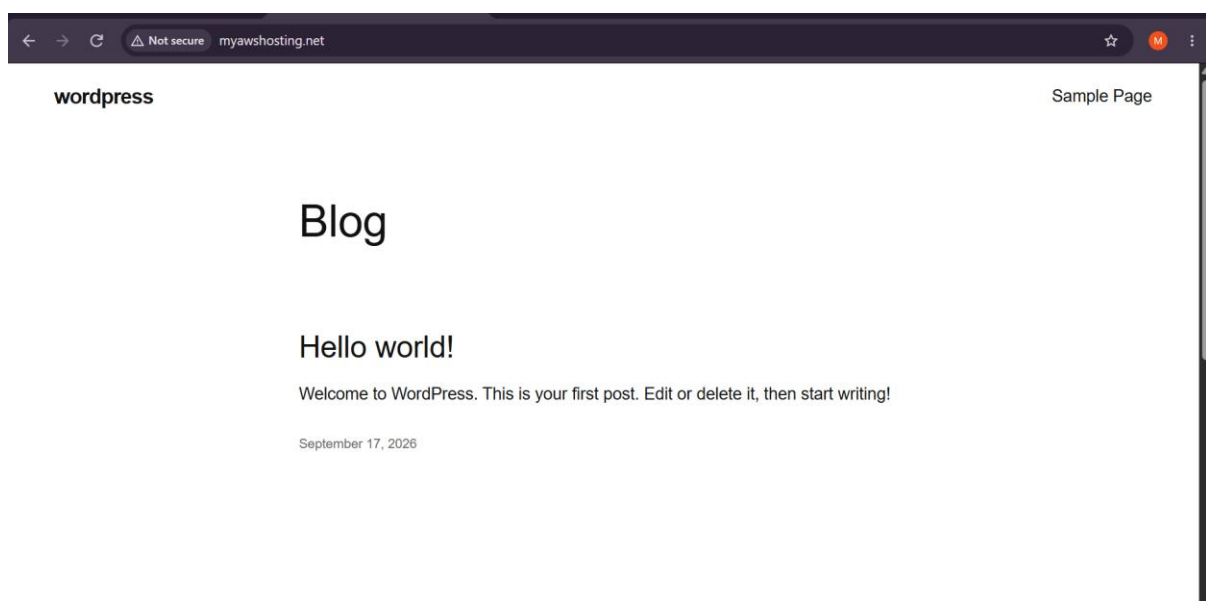
This confirmed that the hostname was successfully reaching the WordPress server.

8.5 Access WordPress Using URL

The website was opened using:

<http://myawshosting.net>

The WordPress website loaded successfully.



9. Security Configuration

The lab was configured using minimum required access.

EC2

- SSH port 22 restricted to the required source.
- HTTP port 80 enabled for website access.
- MySQL port 3306 was not publicly exposed on EC2.

RDS

- Public access disabled.
- MySQL port 3306 allowed only from the EC2 security group.
- RDS was accessed through its private network connection from EC2.

SELinux

The required SELinux permissions were enabled for:

- Apache access to the WordPress directory.
- Apache/PHP network access to the RDS database.

10. Conclusion

This practical successfully demonstrated hosting a WordPress website on AWS using an RHEL EC2 instance and Amazon RDS MySQL.

WordPress was installed under the user directory:

```
/home/ec2-user/wordpress
```

Apache and PHP were configured to serve the WordPress application. An Amazon RDS MySQL database was created with public access disabled, and the EC2 instance was successfully connected to the database.

SELinux was configured to allow Apache/PHP to communicate with the RDS database.

Finally, the hostname:

myawshosting.net

was mapped to the EC2 public IP using the Windows hosts file, and the WordPress website was successfully accessed using:

<http://myawshosting.net>

This completed the AWS WordPress hosting practical.